

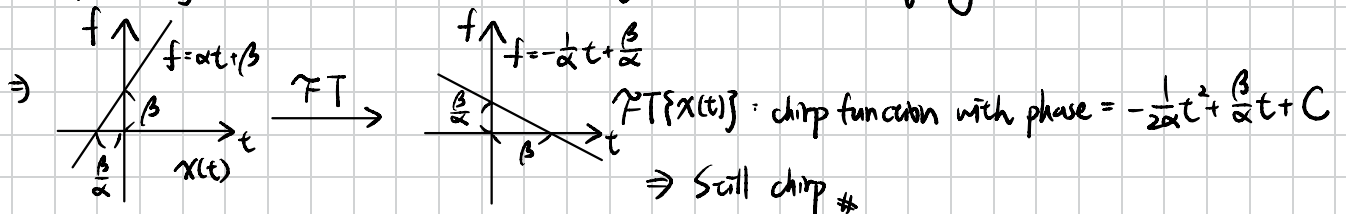
- (1) Suppose that $x(t)$ is a chirp function. Which of the following functions are also chirps? Why? (i) $FT(x(t))$; (ii) the FrFT of $x(t)$; (iii) the generalized modulation of $x(t)$; (iv) $x(t) * x(t)$ where $*$ means the convolution. (Note that the time-frequency distribution of $x(t)$ is a line). (10 scores)

$x(t)$ is a chirp function (phase of order 2)

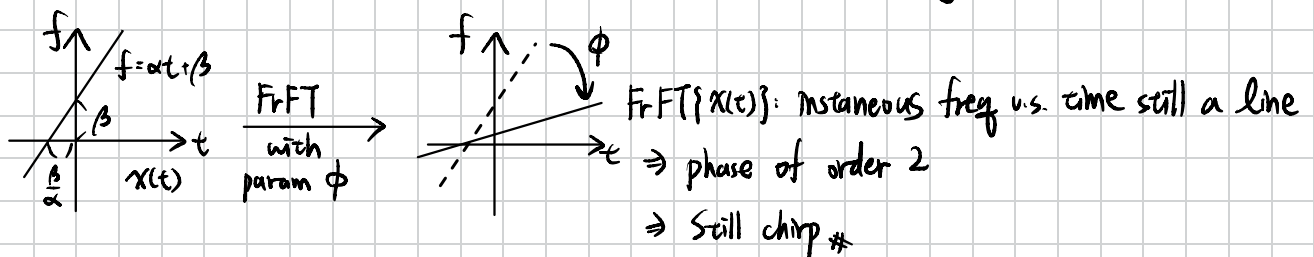
$$\Rightarrow \text{Let } x(t) = A e^{j(\frac{\alpha}{2}t^2 + \beta t + \gamma)}, \quad A \in \mathbb{C}, \alpha, \beta, \gamma \in \mathbb{R}$$

$\Rightarrow$ instantaneous freq: $\frac{d\phi(t)}{dt} = \alpha t + \beta \Rightarrow$ a line in time-frequency distribution

(i) $FT\{x(t)\} \equiv$ do clockwise rotation by 90° to time-frequency distribution

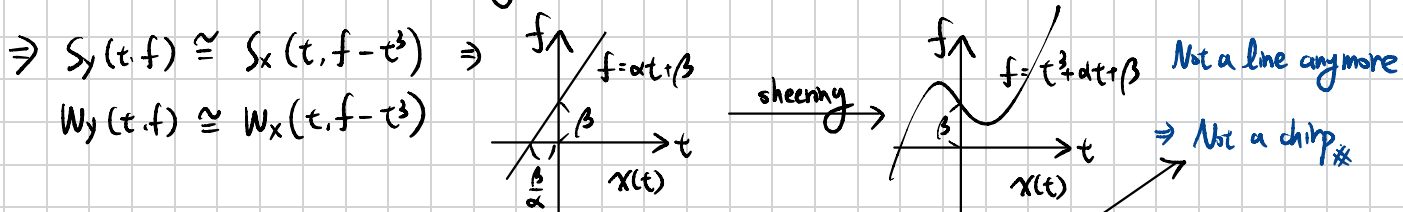


(ii) $\text{FrFT}\{x(t)\} \equiv$ do clockwise rotation with angle ϕ to time-frequency distribution



(iii) Generalized modulation of $x(t)$

Counter example - generalized shearing: $y(t) = e^{j\frac{\lambda}{2}t^4} x(t)$ (take $\phi(t) = \frac{\lambda}{2}t^4$)



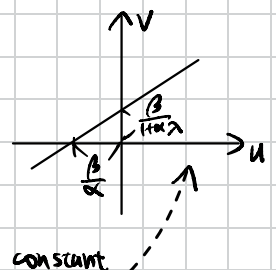
In fact, $y(t) = e^{j\frac{\lambda}{2}t^4} x(t) = A e^{j(t^4 + \frac{\alpha}{2}t^2 + \beta t + \gamma)}$ with phase order > 2

(iv) $x(t) * x(t)$ $\because x(t)$ itself is chirp
 $\equiv x(t)$ convolution with a chirp function on time domain

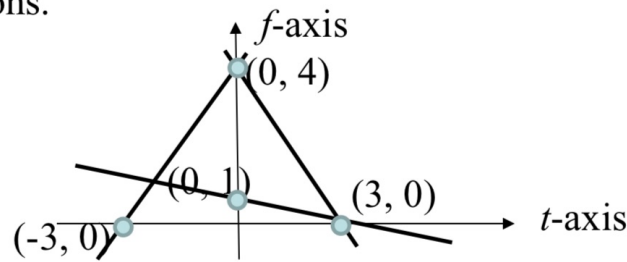
$\Rightarrow$ Fresnel Transform $\Rightarrow$ a specific case of LCT with kernel $\begin{bmatrix} 1 & \lambda \\ 0 & 1 \end{bmatrix}$, λ is constant
($t, f \rightarrow u, v$)

Then initially the line $f = \alpha t + \beta \rightarrow v = \alpha(u - \lambda v) + \beta \Rightarrow v = \frac{\alpha}{1 + \alpha\lambda}u + \frac{\beta}{1 + \alpha\lambda}$ - still a line

$\Rightarrow x(t) * x(t)$ still a chirp function #



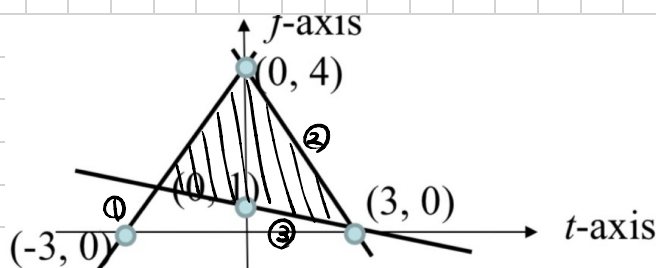
(2) Suppose that the time-frequency distribution of a signal is as the following triangular regions.



Suppose that the signal is interfered by the white noise. How do we use the FrFT filters to reduce the effect of white noise? $H(u)$ and α of each FrFT filter should be shown. (15 scores)

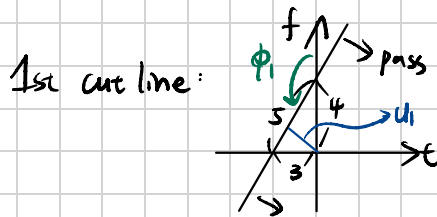
→ ϕ here

White noise: $E[W_X(t, f)] = \delta \Rightarrow$ Literally everywhere



reduce the effect of white noise
||
increase SNR

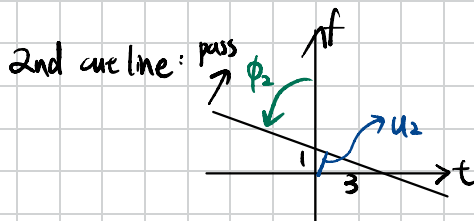
Use 3 x FrFT to design filter (3 cut lines)



$$u_1 = \frac{3 \times 4}{5} = \frac{12}{5}$$

$$\phi_1 = \pi - \tan^{-1}\left(\frac{3}{4}\right)$$

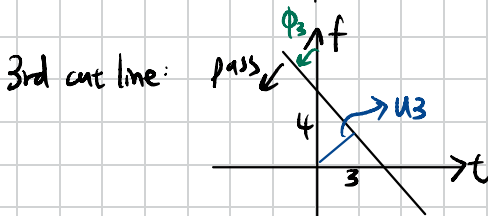
$$\Rightarrow H_1(u) = \begin{cases} 1 & \text{if } u \leq u_1 \\ 0 & \text{if } u > u_1 \end{cases}$$



$$u_2 = \frac{1 \times 3}{\sqrt{10}} = \frac{3}{\sqrt{10}}$$

$$\phi_2 = \tan^{-1}(3)$$

$$\Rightarrow H_2(u) = \begin{cases} 1 & \text{if } u \geq u_2 \\ 0 & \text{if } u < u_2 \end{cases}$$



$$u_3 = \frac{12}{5}$$

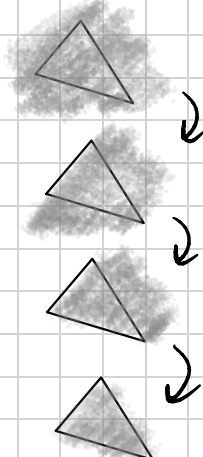
$$\phi_3 = \tan^{-1}\left(\frac{3}{4}\right)$$

$$\Rightarrow H_3(u) = \begin{cases} 1 & \text{if } u \leq u_3 \\ 0 & \text{if } u > u_3 \end{cases}$$

Complete process:

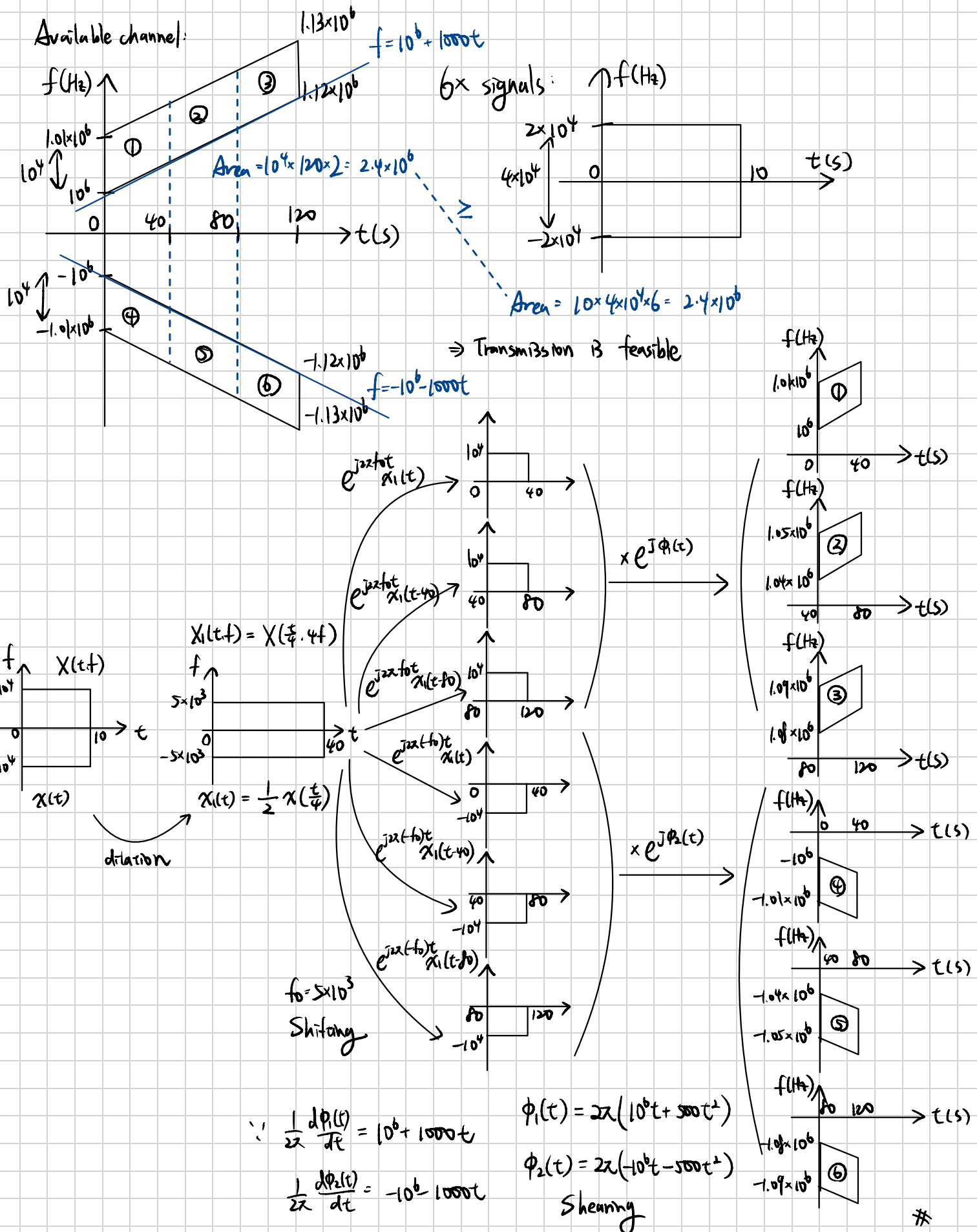
$$\begin{aligned} &X(t) \\ &\downarrow \\ &X_1(t) = O_F^{-\phi_1} (O_F^{\phi_1} (X(t)) H_1(u)) \\ &\downarrow \\ &X_2(t) = O_F^{-\phi_2} (O_F^{\phi_2} (X_1(t)) H_2(u)) \\ &\downarrow \\ &X_3(t) = O_F^{-\phi_3} (O_F^{\phi_3} (X_2(t)) H_3(u)) \end{aligned}$$

*



△: signal
■: white noise

- (3) Suppose that there are 6 signals. Their time length are all 10 seconds and their frequencies are all from $-20000\text{Hz} \sim 20000\text{Hz}$. Also suppose that the channel is almost full except for $f \in [1000000 + 1000t, 1010000 + 1000t]$ and $f \in [-1000000 - 1000t, -1000000 - 1000t]$, $0 < t < 120$. How do we transmit the 6 signals by the channel? (15 scores)



(4) (a) What are the two main differences between the IMF and a sinusoid function?

(b) Which function is an IMF? Why? (i) $\sin(\pi t^3)$; (ii) $(3 - \cos(12\pi t))\cos(\pi t)$

(10 scores)

(a) General form of sinusoidal function: $A \sin(\omega t + \phi)$

① IMF's amplitude can vary with time. while sinusoidal function has fixed amplitude A

② IMF's frequency can vary with time. while sinusoidal function has fixed frequency $\frac{\omega}{2\pi}$

(b) Constraint of IMF:

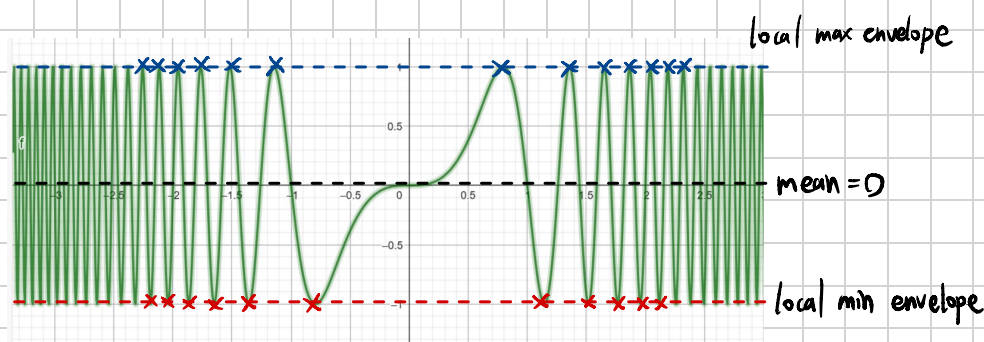
① # of extremes and # of zero-crossings differ at most one

② Mean of local max/min envelopes should near 0

$\pi \cdot \sin(\pi t^3)$

Satisfy both
constraints

$\Rightarrow$ IMF $\#$



$\pi \cdot (3 - \cos(12\pi t)) \cos \pi t$

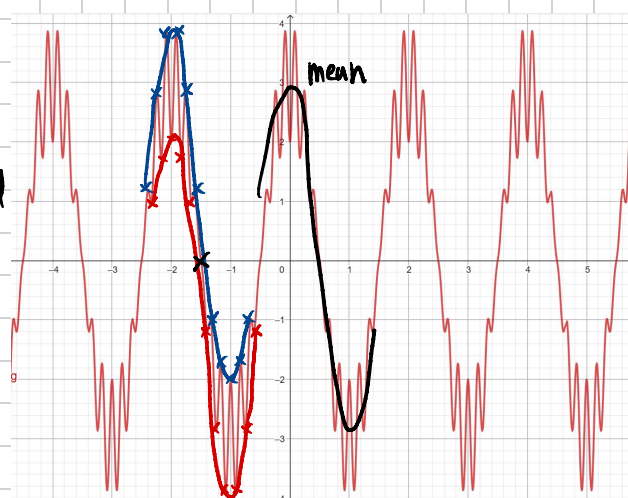
Contradict ①:

of extremes $\approx 22 \times$ # of zero crossings per period

Contradict ②:

mean of local max and min envelope not always
near zero, but follows $3 \cos(\pi t)$ instead.

$\Rightarrow$ not IMF $\#$



(5) (a) What is the most important advantage of the Haar transform nowadays?

(b) Write the 6th row of the 16-point Haar transform.

(10 scores)

(a) Simple and efficient computation is the most important advantage.

① No complex value involved:

The whole process of Haar transform is under real domain. For Fourier Transform, complex multiplication is involved, where complex multiplication $\equiv 4$ real value multiplication

② Literally no multiplication:

As we can see, elements of Haar transform kernel contain only -1, 0, 1. "x1" is to keep the same, "x0" simply assigns zero, while "x(-1)" is only about sign conversion.

$\Rightarrow$ No need multiplier, the computation overhead mainly depends on accumulator and I/O.

In conclusion, we can obtain localized feature of the signal through fast computation thanks to Haar transform. However, with advancement of hardware, Haar Transform's advantage becomes less beneficial since other kinds of transforms become fast enough. *

(b) $H_{16} = \begin{bmatrix} H_8 \otimes [1, 1] \\ I_8 \otimes [1, -1] \end{bmatrix} \Rightarrow 6\text{-th row of } H_{16} = 6\text{-th row of } H_8 \otimes [1, 1]$

$$H_8 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix} \xrightarrow{\otimes [1, 1]} \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

width = 4
 $\Rightarrow$ detect local high frequency *

(6) (a) What is the role of the vanish moment in the wavelet transform?

(b) Suppose that $x(t) = a + bt + t^2$ for $-2 < t < 2$, $x(t) = 0$ otherwise.

If $x(t)$ has the vanish moment of 2, determine a and b . (10 scores)

(a) k -th moment $m_k = \int_{-\infty}^{\infty} t^k \psi(t) dt$, where $\psi(t)$ is mother wavelet.

If $m_0 = m_1 = \dots = m_{p-1} = 0$, $m_p \neq 0 \Rightarrow \psi(t)$ has p vanishing moments.

It's used to identify whether the signal $\psi(t)$ is a high-frequency one.

Reason: $m_k = \langle t^k, \psi(t) \rangle$, $\therefore$ has p vanishing moments $\equiv$ be orthogonal to $t^0, t^1 \sim t^{p-1}$
 $\Rightarrow$ with p higher, $\psi(t)$ has more high-frequency characteristic
 order $\downarrow$, freq $\downarrow$

Since $\psi(t)$ serves as mother wavelet in wavelet transform, when doing $\frac{1}{\sqrt{b}} \int_{-\infty}^{\infty} x(t) \psi(\frac{t-a}{b}) dt$
 if $\psi(t)$ has higher vanishing moments $\Rightarrow$ low frequency components can be filtered out more. *

(b) If $x(t)$ has 2 vanishing moments, $m_0 = m_1 = 0$; $m_2 \neq 0$

$$\Rightarrow m_0 = \int_{-2}^2 x(t) dt = \left. at + \frac{1}{2}bt^2 + \frac{1}{3}t^3 \right|_{-2}^2 = 4a + \frac{16}{3} = 0 \Rightarrow a = -\frac{4}{3}$$

$$m_1 = \int_{-2}^2 t x(t) dt = \left. \frac{1}{2}at^2 + \frac{1}{3}bt^3 + \frac{1}{4}t^4 \right|_{-2}^2 = \frac{16}{3}b = 0 \Rightarrow b = 0$$

$$m_2 = \int_{-2}^2 t^2 x(t) dt = \left. \frac{1}{3}at^3 + \frac{1}{4}bt^4 + \frac{1}{5}t^5 \right|_{-2}^2 = \frac{16}{3}a + \frac{64}{5} \neq 0 \quad a = -\frac{4}{3}, b = 0 *$$

(7) Write a Matlab or Python program of the Hilbert-Huang transform.

$$y = \text{hht}(x, t, \text{thr})$$

x : input, y : output (each row of y is one of the IMFs of x), t : samples on the t -axis, thr : the threshold used in Step 7.

In Step 8, the number of non-boundary extremes can be no more than 3.

Step 9 is not required. The code should be handed out by NTUCool.

(30 scores)

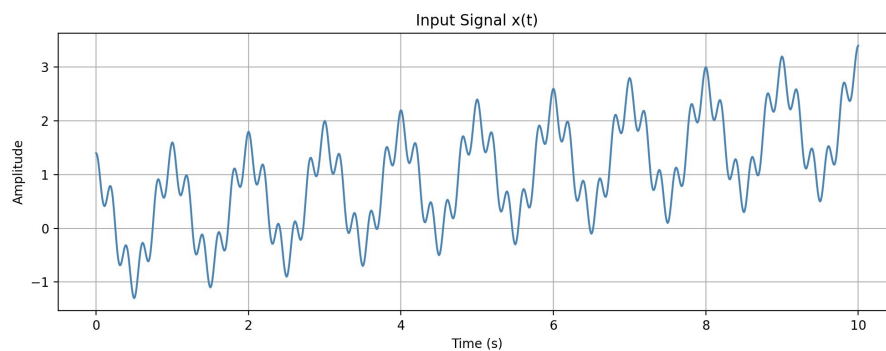
Example: $t = [0: 0.01: 10];$

$$x = 0.2*t + \cos(2*\pi*t) + 0.4*\cos(10*\pi*t);$$

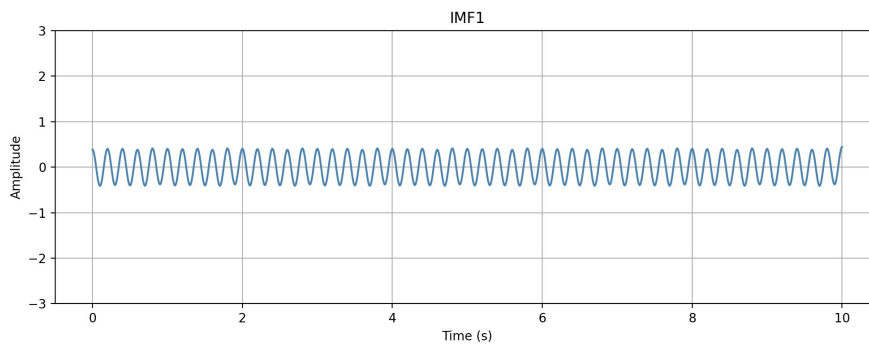
$$\text{thr} = 0.2;$$

$$y = \text{hht}(x, t, \text{thr});$$

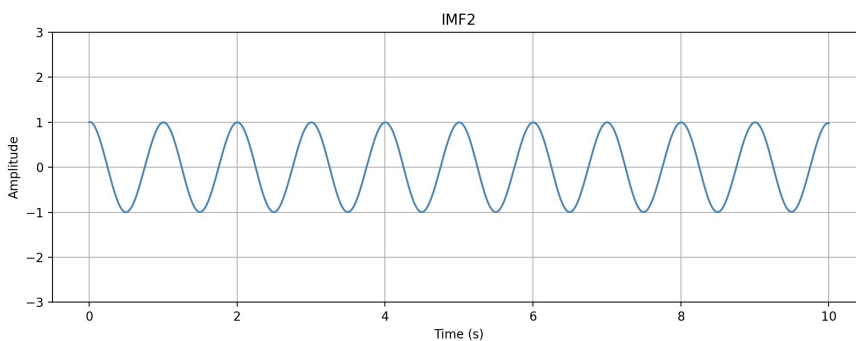
Input:



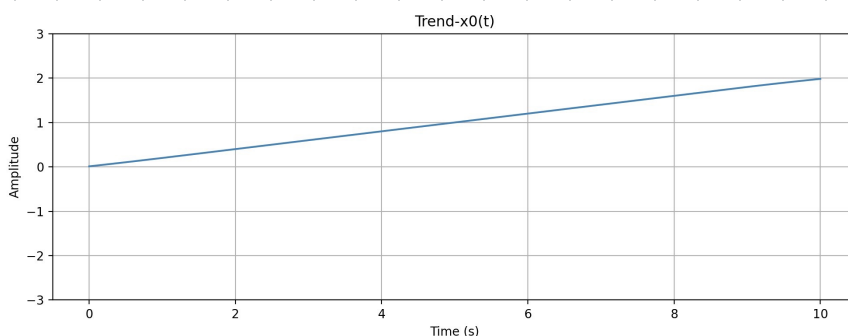
IMF1



IMF2



$x_0(t)$
Trend



Code Screenshots:

Show Signal:

```
HHTF.py > find_extreme
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time
4 import scipy.interpolate as interpolate
5
6 try:
7     from test_complex_signal import generate_complex_signal
8     print("Successfully import complex signal")
9     complexFlag = True
10 except ImportError:
11     complexFlag = False
12     print("Cannot find complex signal module")
13
14 np.set_printoptions(threshold=10)
15
16 def show_signal(x, t, title="Input Signal", y_max = 3, save_fig = False):
17     plt.figure(figsize=(10, 4))
18     plt.plot(t, x, color='steelblue')
19     plt.title(title)
20     plt.xlabel('Time (s)')
21     plt.ylabel('Amplitude')
22     plt.ylim(-y_max, y_max)
23     plt.grid(True)
24     plt.tight_layout()
25     if save_fig:
26         plt.savefig(f"{title}.png", dpi=200)
27     plt.show()
28     plt.close()
29     return
```

Find extremes:

```
1 def find_extreme(x, t, endpoint = True):
2     dx = np.diff(x)
3     print(len(dx))
4     print(dx)
5     signs = np.sign(dx)
6     sign_change = np.diff(signs)
7     max_idx = np.where(sign_change < 0)[0] + 1
8     min_idx = np.where(sign_change > 0)[0] + 1
9
10    # For end points:
11    extreme_idx = np.sort(np.concatenate([max_idx, min_idx]))
12    if(endpoint):
13        endpoints = [0, len(x) - 1]
14
15    # For right endpoint: x[0]
16    d10 = abs(extreme_idx[0])
17    d21 = abs(extreme_idx[1]-extreme_idx[0])
18
19    if d10 > 0.5 * d21:
20        # as extreme
21        if x[0] > x[1]:
22            max_idx = np.insert(max_idx, 0, 0)
23        else:
24            min_idx = np.insert(min_idx, 0, 0)
25    # For Left endpoint: x[len(x)-1]
26    d10_ = abs(len(x)-1-extreme_idx[-1])
27    d21_ = abs(extreme_idx[-1]-extreme_idx[-2])
28
29    if d10_ > 0.5 * d21_:
30        # as extreme
31        if x[-1] > x[-2]:
32            max_idx = np.append(max_idx, len(x)-1)
33        else:
34            min_idx = np.append(min_idx, len(x)-1)
35
36    return max_idx, min_idx, extreme_idx
```

Find envelopes:

```
68 def find_upper_bottom_envelop(x, t):
69     dt = t[1]-t[0]
70     local_max, local_min, _ = find_extreme(x, t)
71     print("local max:", local_max)
72     print("local_min:", local_min)
73
74     if len(local_max) > 3:
75         t_, c, k = interpolate.splrep(local_max * dt, x[local_max], k = 3) # k: degree
76         f = interpolate.BSpline(t_, c, k)
77         emax = f(t)
78     else:
79         emax = np.interp(t, t[local_max], x[local_max])
80     #show_signal(emax, t, "Local peaks")
81     if len(local_min) > 3:
82         t_, c, k = interpolate.splrep(local_min * dt, x[local_min], k = 3)
83         f = interpolate.BSpline(t_, c, k)
84         emin = f(t)
85     else:
86         emin = np.interp(t, t[local_min], x[local_min])
87     return emax, emin, local_max, local_min
```

check IMF:

```
89 def checkIMF(x, t, threshold):
90     u1, u0, local_max, local_min = find_upper_bottom_envelop(x, t)
91     posflag = np.all(x[local_max]>0)
92     if not posflag:
93         return False
94     negflag = np.all(x[local_min]<0)
95     if not negflag:
96         return False
97     lessflag = np.all(np.abs((u1+u0)/2)<threshold)
98     if not lessflag:
99         return False
100    return True
```

Hilbert - Huang Transform

```
102 def hht(x, t, thr, K = 10):
103     c = [] # List to store IMF
104     findtrend = False
105     n = 0
106     y = x
107     while not findtrend:
108         print(f"n={n}")
109         isIMF = False
110         k = 0
111         while (not isIMF):
112             print(f"k={k}")
113             emax, emin, _, _ = find_upper_bottom_envelop(y, t)
114             z = (emax+emin)/2
115             h = y - z
116             if(checkIMF(h, t, thr)):
117                 isIMF = True
118             else:
119                 y = h
120                 k = k + 1
121         c.append(h) # Append (n+1)-th IMF
122         x0 = x - sum(c)
123         _, _, extreme_index = find_extreme(x0, t, endpoint=False) #Not include endpoint
124         if len(extreme_index) <= 3: # trend should be polynomial of order <= 4
125             findtrend = True
126         else:
127             n = n + 1
128             y = x0
129     return c, x0
```

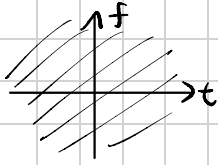
Main Function

```
132 if __name__ == "__main__":
133     dt = 0.01 # Δt = 0.05
134     t = np.arange(0, 10+dt, dt)
135     verbose = True
136     thr = 0.2
137     x = 0.2 * t + np.cos(2 * np.pi * t) + 0.4 * np.cos(10 * np.pi * t)
138     #kinds = ['multi_close', 'chirp+noise', 'am_fm', 'intermittent',
139             'trend_nonlin', 'hetero_noise', 'impulses', 'complex_all']
140     #x = generate_complex_signal(kinds[7], t, seed=42)
141     # ===== show input signal =====
142     plt.figure(figsize=(10, 4))
143     plt.plot(t, x, color='steelblue')
144     plt.title('Input Signal x(t)')
145     plt.xlabel('Time (s)')
146     plt.ylabel('Amplitude')
147     plt.grid(True)
148     plt.tight_layout()
149     plt.savefig("input_signal.png", dpi=200)
150     plt.show()
151     plt.close()
152     # =====
153
154     # calculate Hilbert Huang Transform
155
156     # ==== Execution time start ====
157     start_time = time.time()
158     IMFs, x0 = hht(x, t, thr)
159     # ==== Execution time end ====
160     comp_time = time.time() - start_time
161     n = 1
162     for imf in IMFs:
163         show_signal(imf, t, f"IMF{n}", save_fig = True)
164         n = n + 1
165     show_signal(x0, t, f"Trend-x0(t)", save_fig = True)
166
167     print(f"Computation time = {comp_time:.6f} s")
```

(Extra): Answer the questions according to your student ID number.

(ended with 0, 2, 3, 4, 5, 7, 8, 9)

If $x(t)$ is white noise, which functions are also white?



$$E\{W_x(t, f)\} = \delta = \text{const.}$$

(iv) $y(t) = \text{FT}\{x(t)\}$ $\Rightarrow$ clockwise rotation by 90° , $W_y(t, f) = W_x(-f, t)$

$\Rightarrow E\{W_y(t, f)\} = \delta$ as well $\Rightarrow$ also white noise

(v) $y(t) = \text{FrFT}\{x(t)\}$ $\Rightarrow$ clockwise rotation by ϕ , $W_y(t, f) = W_x(t \cos \phi - f \sin \phi, t \sin \phi + f \cos \phi)$

$\Rightarrow E\{W_y(t, f)\} = \delta$ as well $\Rightarrow$ also white noise

(vi) $y(t) = x(t - t_0)$ $\Rightarrow$ shift horizontally, $W_y(t, f) = W_x(t - t_0, f)$

$\Rightarrow E\{W_y(t, f)\} = \delta$ as well $\Rightarrow$ also white noise *